Summer Internship 2026

Advanced Full Stack Development

Day-wise Curriculum & Practice Guide

Duration	**40 Days (8 Weeks)**	**Hours/Day**	**5 Hours**
Stack	**React · Node · MongoDB**	**Format**	**Online / Lab**

Week	Days	Focus Area	Key Technologies
Week 1	1–5	**Advanced JavaScript**	ES6+ Foundations, Closures, Scope & Higher-Order Functions…
Week 2	6–10	**Async JavaScript & APIs**	Callbacks & the Event Loop, Promises…
Week 3	11–15	**React Fundamentals**	React Intro & JSX, Components & Props…
Week 4	16–20	**Advanced React**	React Router, Context API & Global State…
Week 5	21–25	**Node.js & Express.js**	Node.js Fundamentals, Express.js Basics…
Week 6	26–30	**MongoDB & Authentication**	MongoDB & Mongoose Basics, Advanced Mongoose…
Week 7	31–35	**Full Stack Integration**	Connecting React to Express, Full Stack Auth Flow…
Week 8	36–40	**Deployment & Final Presentation**	Deploying the Backend, Deploying the Frontend…

Week 1 · Advanced JavaScript (1–5 Days)

Day 1 ES6+ Foundations

Topics Covered

- • let, const vs var – scope & hoisting
- • Arrow functions & implicit return
- • Template literals & tagged templates
- • Destructuring: arrays & objects
- • Default parameters & rest/spread operator

■ Practice Task

Refactor old ES5 code snippets using ES6+ syntax; write 10 mini-functions using arrow functions.

■ Learning Outcome

Write clean modern JavaScript confidently.

Day 2 Closures, Scope & Higher-Order Functions

Topics Covered

- • Lexical scope & closure mechanics
- • IIFE pattern
- • map(), filter(), reduce() deep-dive
- • Function composition & currying basics
- • Memoization concept

■ Practice Task

Build a shopping-cart module using closures; implement custom map/filter/reduce from scratch.

■ Learning Outcome

Understand functional programming fundamentals.

Day 3 Prototypes, Classes & OOP

Topics Covered

- • Prototype chain & Object.create()
- • ES6 class syntax, constructor, methods
- • Inheritance with extends & super
- • Static methods & getters/setters
- • Mixins pattern

■ Practice Task

Model a Library system with Book, Member, and Librarian classes using inheritance.

■ Learning Outcome

Design object-oriented solutions in JavaScript.

Day 4 Error Handling & Modules

Topics Covered

- • try / catch / finally & custom Error classes
- • ES Modules: import / export (named & default)
- • CommonJS require vs ESM
- • Dynamic import() for lazy loading
- • Module bundling concept (why Webpack/Vite?)

■ Practice Task

Split a utility library into modules; add proper error handling with custom errors.

■ Learning Outcome

Structure JavaScript code with modules and handle errors gracefully.

Day 5 DOM Manipulation & Events

Topics Covered

- • querySelector, querySelectorAll, DOM traversal
- • createElement, appendChild, innerHTML vs textContent
- • Event listeners, bubbling & capturing
- • Event delegation pattern
- • LocalStorage & SessionStorage

■ Practice Task

Build an interactive To-Do list (add, delete, mark done) using only vanilla JS & DOM APIs.

■ Learning Outcome

Manipulate the DOM dynamically without any library.

Week 2 · Async JavaScript & APIs (6–10 Days)

Day 6 Callbacks & the Event Loop

Topics Covered

- • Synchronous vs asynchronous execution
- • Call stack, Web APIs, Callback queue
- • Event loop mechanics (visual walkthrough)
- • Callback pattern & callback hell
- • setTimeout / setInterval internals

■ Practice Task

Simulate an async waterfall using callbacks; visualize event loop with console.log order exercises.

■ Learning Outcome

Explain how JavaScript handles async tasks under the hood.

Day 7 Promises

Topics Covered

- • Promise states: pending, fulfilled, rejected
- • .then(), .catch(), .finally()
- • Promise chaining
- • Promise.all(), Promise.allSettled(), Promise.race()
- • Converting callbacks to Promises

■ Practice Task

Fetch 3 APIs in parallel using Promise.all(); implement a retry-with-delay mechanism.

■ Learning Outcome

Write clean async code with Promises.

Day 8 Async / Await

Topics Covered

- • async function & await keyword
- • Error handling with try/catch inside async
- • Sequential vs parallel execution
- • Top-level await (ESM)
- • Async iteration (for await...of)

■ Practice Task

Rewrite yesterday's Promise chain using async/await; build a weather fetcher with error fallback.

■ Learning Outcome

Write readable async code that looks synchronous.

Day 9 Fetch API & REST Concepts

Topics Covered

- • HTTP verbs: GET, POST, PUT, PATCH, DELETE
- • fetch() with headers, body, method
- • Parsing JSON responses
- • REST API conventions & status codes
- • CORS concept & why it matters

■ Practice Task

Consume the JSONPlaceholder API – fetch, create, update, and delete posts using fetch().

■ Learning Outcome

Communicate with any REST API from the browser.

Day 10 Axios & API Project

Topics Covered

- • Axios vs fetch – why Axios?
- • Axios interceptors for auth tokens
- • Query params & base URL config
- • Error handling with Axios
- • Building a reusable API service layer

■ Practice Task

Build a GitHub Profile Finder – search a username, show repos, followers using GitHub API.

■ Learning Outcome

Build a complete API-driven mini-app.

Week 3 · React Fundamentals (11–15 Days)

Day 11 React Intro & JSX

Topics Covered

- • Why React? Virtual DOM concept
- • Create React App / Vite setup
- • JSX syntax rules & expressions
- • Rendering elements & React.createElement
- • Project structure best practices

■ Practice Task

Set up Vite + React project; build a static Profile Card component with JSX.

■ Learning Outcome

Set up a React project and write JSX confidently.

Day 12 Components & Props

Topics Covered

- • Functional components
- • Props – passing & receiving
- • PropTypes for validation
- • children prop & composition
- • Component reusability principles

■ Practice Task

Build a reusable Card component; render a product listing by passing different props.

■ Learning Outcome

Design reusable components driven by props.

Day 13 State & useState Hook

Topics Covered

- • State vs props difference
- • useState – declaring, reading, updating
- • State with objects & arrays
- • Immutability principle
- • Lifting state up

■ Practice Task

Build a Counter App; extend into a Shopping Cart with add/remove/quantity update.

■ Learning Outcome

Manage local component state with useState.

Day 14 useEffect & Lifecycle

Topics Covered

- • useEffect syntax & dependency array
- • Run on mount, update, unmount
- • Cleanup function
- • Fetching data with useEffect
- • Common useEffect mistakes

■ Practice Task

Fetch and display a list of posts from an API on component mount; add a search-filter effect.

■ Learning Outcome

Handle side effects and async data in React.

Day 15 Events, Forms & Conditional Rendering

Topics Covered

- • Synthetic events & onClick, onChange
- • Controlled vs uncontrolled inputs
- • Form submission & validation
- • Conditional rendering: &&, ternary, if/else
- • List rendering with .map() & keys

■ Practice Task

Build a Login/Register form with validation; conditionally show dashboard or login screen.

■ Learning Outcome

Handle user interactions and forms in React.

Week 4 · Advanced React (16–20 Days)

Day 16 React Router

Topics Covered

- • react-router-dom v6 setup
- • BrowserRouter, Routes, Route
- • Link, NavLink, useNavigate
- • Dynamic routes & useParams
- • Nested routes & Outlet

■ Practice Task

Add multi-page routing to your app: Home, About, Products, Product Detail pages.

■ Learning Outcome

Build multi-page React applications with routing.

Day 17 Context API & Global State

Topics Covered

- • Prop drilling problem
- • createContext & Provider
- • useContext hook
- • Theme switcher with Context
- • Auth context pattern

■ Practice Task

Implement dark/light theme toggle; add an AuthContext for user login state across pages.

■ Learning Outcome

Share state globally without prop drilling.

Day 18 Advanced Hooks

Topics Covered

- • useReducer for complex state
- • useRef – DOM access & persistent values
- • useMemo & useCallback – performance
- • Custom hooks – building & using
- • useId, useTransition (React 18)

■ Practice Task

Refactor Shopping Cart to useReducer; build a custom useFetch hook used across 3 components.

■ Learning Outcome

Write powerful custom hooks and optimize performance.

Day 19 State Management with Redux Toolkit

Topics Covered

- • Redux concepts: store, action, reducer
- • Redux Toolkit: createSlice, configureStore
- • useSelector & useDispatch
- • Async thunks with createAsyncThunk
- • Redux DevTools

■ Practice Task

Migrate Cart Context to Redux Toolkit; add an async product fetch thunk.

■ Learning Outcome

Manage large-scale application state with Redux.

Day 20 Multi-page App Project

Topics Covered

- • Component architecture planning
- • Protected routes (auth guard)
- • Lazy loading with React.lazy & Suspense
- • React.memo for optimization
- • Error boundaries

■ Practice Task

Build a full E-Commerce UI – Home, Products, Cart, Login with protected routes & Redux.

■ Learning Outcome

Complete a production-quality multi-page React application.

Week 5 · Node.js & Express.js (21–25 Days)

Day 21 Node.js Fundamentals

Topics Covered

- • What is Node.js? Event-driven, non-blocking I/O
- • Node REPL & running scripts
- • Core modules: fs, path, os, events
- • npm & package.json
- • CommonJS vs ESM in Node

■ Practice Task

Build a CLI file manager – list, read, write, delete files using fs module.

■ Learning Outcome

Write server-side JavaScript with Node.js core APIs.

Day 22 Express.js Basics

Topics Covered

- • Setting up an Express app
- • Routing: app.get, app.post, app.put, app.delete
- • Route parameters & query strings
- • Sending JSON responses & status codes
- • Postman for API testing

■ Practice Task

Build a basic Express server with CRUD routes for a 'books' resource; test with Postman.

■ Learning Outcome

Create a RESTful API server with Express.

Day 23 Middleware

Topics Covered

- • What is middleware? next() function
- • Built-in: express.json(), express.static()
- • Third-party: cors, morgan, helmet
- • Custom middleware (logger, auth check)
- • Error-handling middleware

■ Practice Task

Add logging, CORS, and a custom auth-check middleware to your books API.

■ Learning Outcome

Extend Express behaviour with middleware layers.

Day 24 REST API Design & Validation

Topics Covered

- • RESTful conventions & resource naming
- • Input validation with express-validator / Joi
- • Consistent API response format
- • HTTP status codes best practices
- • API versioning (/api/v1/)

■ Practice Task

Redesign the books API with versioning, Joi validation, and standardised JSON responses.

■ Learning Outcome

Design clean, production-grade REST APIs.

Day 25 File Upload & Environment Config

Topics Covered

- • dotenv & .env file management
- • File upload with multer
- • Serving static files
- • Express Router – modular routing
- • Rate limiting with express-rate-limit

■ Practice Task

Add an image-upload endpoint to the API; split routes into separate Router files.

■ Learning Outcome

Handle files and configure environment variables securely.

Week 6 · MongoDB & Authentication (26–30 Days)

Day 26 MongoDB & Mongoose Basics

Topics Covered

- • NoSQL vs SQL – when to use which
- • MongoDB Atlas setup & connection string
- • Mongoose connect, Schema, Model
- • CRUD with Mongoose: find, save, update, delete
- • Data types & validation in Schema

■ Practice Task

Replace in-memory books array with MongoDB; implement all CRUD operations with Mongoose.

■ Learning Outcome

Persist data in MongoDB using Mongoose ODM.

Day 27 Advanced Mongoose

Topics Covered

- • Schema references & populate (relations)
- • Virtuals & instance methods
- • Query chaining: select, sort, limit, skip
- • Pagination implementation
- • Mongoose pre/post hooks (middleware)

■ Practice Task

Add Author model; populate author details in book queries; implement pagination.

■ Learning Outcome

Model relationships and query data efficiently.

Day 28 Authentication with JWT

Topics Covered

- • Auth concepts: session vs token-based
- • bcryptjs for password hashing
- • JWT structure: header.payload.signature
- • Signing & verifying tokens
- • Auth middleware to protect routes

■ Practice Task

Add User model; implement /register and /login routes; protect book routes with JWT middleware.

■ Learning Outcome

Implement secure user authentication from scratch.

Day 29 Authorization & Role-Based Access

Topics Covered

- • Authentication vs authorization
- • Role field in User model (admin, user)
- • Role-check middleware
- • Resource ownership check
- • Refresh tokens concept

■ Practice Task

Add admin-only delete/update; users can only edit their own records; test all scenarios in Postman.

■ Learning Outcome

Implement role-based access control (RBAC).

Day 30 CRUD + Auth Complete App

Topics Covered

- • Consolidate all routes: auth + resources
- • Password reset flow (email token concept)
- • API documentation with Swagger/Postman collection
- • Security best practices: helmet, sanitise
- • Code review & refactoring

■ Practice Task

Finalize a complete Blog API with auth, roles, posts, comments, and Postman collection.

■ Learning Outcome

Deliver a production-ready authenticated REST API.

Week 7 · Full Stack Integration (31–35 Days)

Day 31 Connecting React to Express

Topics Covered

- • Proxy setup in Vite/CRA for dev
- • Axios instance with base URL & interceptors
- • Storing JWT in memory vs localStorage
- • Sending Authorization header
- • Handling 401 errors globally

■ Practice Task

Connect the React E-Commerce UI to the Express backend; implement login flow end-to-end.

■ Learning Outcome

Link a React frontend to a Node/Express backend.

Day 32 Full Stack Auth Flow

Topics Covered

- • Register/Login from React form → API → DB
- • Persist login with token in Context/Redux
- • Protected routes on frontend
- • Logout & token invalidation
- • Auto-login on refresh with stored token

■ Practice Task

Implement full auth cycle: Register → Login → Protected Dashboard → Logout.

■ Learning Outcome

Build a complete authentication flow across the stack.

Day 33 CRUD From Frontend

Topics Covered

- • Create/Read/Update/Delete from React UI
- • Optimistic UI updates
- • Loading & error state management
- • Form handling for API calls
- • Image upload from React to Express

■ Practice Task

Build product management: add, edit, delete products from the React UI connected to DB.

■ Learning Outcome

Perform full CRUD operations from the frontend.

Day 34 Search, Filter & Pagination

Topics Covered

- • Server-side search with query params
- • Debouncing search input
- • Filter by category / price range
- • Pagination component
- • Infinite scroll concept

■ Practice Task

Add search, category filter, and paginated product listing to the full-stack app.

■ Learning Outcome

Implement real-world search & pagination features.

Day 35 Full Stack Project Integration Day

Topics Covered

- • Review & polish all features
- • Fix bugs & edge cases
- • Add loading skeletons & empty states
- • Responsive design check (mobile-first)
- • Git workflow: branching & commits

■ Practice Task

Finalize and commit the complete full-stack project to GitHub with a proper README.

■ Learning Outcome

A fully integrated, well-documented full-stack application.

Week 8 · Deployment & Final Presentation (36–40 Days)

Day 36 Deploying the Backend

Topics Covered

- • Environment variables for production
- • Deploy Node/Express to Render / Railway
- • Connect production MongoDB Atlas
- • CORS configuration for production domains
- • Health-check endpoint

■ Practice Task

Deploy the Express backend to Render; verify all API endpoints are live.

■ Learning Outcome

Deploy a Node.js backend to a cloud platform.

Day 37 Deploying the Frontend

Topics Covered

- • Production build: npm run build
- • Deploy React app to Vercel / Netlify
- • Environment variables in Vite (.env)
- • Custom domain & HTTPS basics
- • CI/CD concept with GitHub Actions

■ Practice Task

Deploy the React frontend to Vercel; point it to the live backend API.

■ Learning Outcome

Deploy a React application to a CDN-based platform.

Day 38 Performance & Best Practices

Topics Covered

- • Lighthouse audit & Core Web Vitals
- • Code splitting & lazy loading
- • Image optimisation (WebP, lazy load)
- • API response caching basics
- • Security checklist: HTTPS, sanitise, rate limit

■ Practice Task

Run Lighthouse on deployed app; fix at least 3 performance issues; re-audit.

■ Learning Outcome

Optimise and secure a deployed full-stack application.

Day 39 Portfolio & GitHub Profile

Topics Covered

- • Writing a great README (badges, screenshots, live link)
- • GitHub profile README
- • Pinning repositories
- • Writing a project case study
- • LinkedIn project post tips

■ Practice Task

Write a full README for the project; update GitHub profile; add live links.

■ Learning Outcome

Present your work professionally to recruiters.

Day 40 Final Presentation & Demo Day

Topics Covered

- • Prepare a 5-minute project demo
- • Explain technical decisions
- • Handle Q&A; from panel
- • Peer code review session
- • Internship wrap-up & next steps

■ Practice Task

Present the final full-stack project to the panel; give & receive peer feedback.

■ Learning Outcome

Confidently demo a full-stack project to an audience.

■ Certification & Benefits

■ **Internship Certificate**	Awarded on 80% attendance & project submission.
■ **Real-World Projects**	3+ deployable projects for your portfolio.
■ **Live Deployment**	Frontend on Vercel, Backend on Render – live URLs.
■ **GitHub Portfolio**	Professional README, commits & pinned repos.
■ **Placement Support**	Resume review, mock interviews & referrals.

Built with dedication · Advanced Full Stack Internship 2026